

Practical Problems: Coding Basics

Learning Outcome Addressed

- Interpret and write code using basic data types and operations.
- Interpret and write code using variables.
- Recognize error messages in Python code.
- Demonstrate understanding of Python's core syntax rules.
- Demonstrate understanding of the difference between expressions and statements.
- Translate simple algorithms to Python functions.

This assignment brings Module 1 together and allows you to apply what you have learned. Follow the instructions below to complete problems one to eight. If you get stuck and are not getting anywhere, please reach out to the learning facilitator for assistance.

1. [15 pts] box_volume

Write the function **box_volume** that takes in three positive integers that represent the height, width, and length of a box respectively, and returns the volume of that box.

As a reminder, the volume of a box is found by multiplying together the height, width, and length.

2. [15 pts] make_introduction

Write the function **make_introduction** that takes in two strings representing a name and hobby and returns the sentence `My name is {name} and I like {hobby}`

3. [25 pts] find_root

Write the function **find_root** that takes in integers a, b, c, and returns the positive quadratic root of the equation

$$ax^2 + bx + c = 0$$

using the quadratic equation.

As a reminder, the quadratic equation is

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

. You only need to return the positive root.

4. [15 pts] Fix the syntax error

Uncomment the code under Question 4 and fix the syntax error(s) so that the test cases pass. The function should correctly return the tens digit of the inputted integer n.

You should modify the function as little as possible. In other words, do not just rewrite it from scratch.

5. [15 pts] Fix the runtime error

Fix the runtime error(s). The function **compute_total** should return the total amount after the inputted tax amount is added.

You should modify the function as little as possible. In other words, do not just rewrite it from scratch.

6. [15 pts] Fix the logical error

Fix the logical error(s) in the function **slope**, which should calculate the slope between two points specified by the (x1, y1) and (x2, y2) coordinates.

As a reminder, slope is calculated as rise/run, where rise is the difference in the y-values and run is the difference in x-values.

You should modify the function as little as possible. In other words, do not just rewrite it from scratch.

7. [25 pts] nth_fibonacci_number

Write the function **nth_fibonacci_number** that takes in an integer n and returns the n th fibonacci number.

This website (<https://math.hmc.edu/funfacts/fibonacci-number-formula/>) describes the algorithm to do this which is given below.

Fibonacci Number Formula

The Fibonacci numbers are generated by setting $F_0 = 0$, $F_1 = 1$, and then using the recursive formula

$$F_n = F_{n-1} + F_{n-2}$$

to get the rest. Thus the sequence begins: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... This sequence of Fibonacci numbers arises all over mathematics and also in nature.

However, if I wanted the 100th term of this sequence, it would take lots of intermediate calculations with the recursive formula to get a result. Is there an easier way?

Yes, there is an exact formula for the n -th term! It is:

$$a_n = [\text{Phi}^n - (\text{phi})^n] / \text{Sqrt}[5].$$

where $\text{Phi} = (1 + \text{Sqrt}[5]) / 2$ is the so-called golden mean, and

$\text{phi} = (1 - \text{Sqrt}[5]) / 2$ is an associated golden number, also equal to $(-1 / \text{Phi})$. This formula is attributed to Binet in 1843, though known by Euler before him.

8. [25 pts] Fix the Errors

Uncomment the function for Question 8 and fix the errors so that it passes the test cases. The function **number_bills_needed** should take in a total cost and return the lowest number of 20, 5, and \$1 bills needed to cover this cost.

You should modify the function as little as possible. In other words, do not just rewrite it from scratch.